

CSCE 713 SOFTWARE SECURITY · MODULE 03 · LECTURE NOTES

Integers and Types

Arithmetic that does not behave like arithmetic.

MODULE OUTCOMES

- Explain overflow, truncation, and sign confusion.
- Explain integer promotion and its surprises.
- Explain why these become memory safety defects.
- Explain type confusion and its mechanism.
- Write arithmetic that cannot overflow silently.

1 The three failures

Failure	The mechanism	The typical consequence
Overflow	The arithmetic result exceeds the type's range and wraps around.	A small allocation followed by a large copy — the canonical pattern below.
Truncation	A value assigned to a narrower type; the high bits are discarded.	A large length silently becomes a small one, after which the check on the small value passes.
Sign confusion	A signed value reinterpreted as unsigned, or the reverse.	A negative length becomes an enormous positive one — and negative lengths arrive from input more often than you would expect.

```
size_t n = read_count_from_input();
buf = malloc(n * sizeof(item)); // may overflow
for (i = 0; i < n; i++)
    buf[i] = read_item(); // writes past end
```

WHAT HAPPENS

n is large enough that `n * sizeof(item)` wraps
 malloc succeeds with a SMALL allocation
 the loop then writes n items into it

AND THE CHECK THAT DOES NOT HELP

```
if (n * sizeof(item) > LIMIT) return;
-- the multiplication overflows BEFORE the
   comparison, so the check inspects the wrapped
   value and passes happily
```

THE FIX: check the operands, not the result.

```
if (n > LIMIT / sizeof(item)) return;
or use a checked-arithmetic helper that reports
the overflow rather than wrapping.
```

'Check the operands, not the result' is the whole rule — because by the time you can see the result, it has already been corrupted, and no amount of validation applied to a wrapped value recovers the information that was lost. This slide is the module's single most useful artefact, and the pattern it describes accounts for a large share of the heap overflows in the published defect data.

2 Promotion, and the rules that surprise people

C PROMOTES SMALL TYPES TO INT, AND COMPARES SIGNED AGAINST UNSIGNED BY CONVERTING TO UNSIGNED

Any arithmetic type smaller than int is promoted to int before arithmetic is performed — so char and short arithmetic actually happens at int width, and overflows at a different point than the declared type suggests.

And in a comparison between a signed and an unsigned type of the same width, the signed operand is converted to unsigned — so `-1 < 1u` evaluates to *false*, because `-1` becomes the largest unsigned value. This is correct per the standard and surprises nearly everybody.

Which breaks the obvious defensive check. `if (len < 0 || len > max)` with an unsigned len has a first clause that can never be true, and which the compiler may therefore remove entirely — frequently with a warning nobody read.

And signed overflow is undefined behaviour in C and C++, which means the compiler is permitted to optimise on the assumption that it cannot happen — and this has removed real overflow checks from real code, because a check written as `if (a + b < a)` is provably false under that assumption and gets deleted. Which is why the checked builtins exist.

- **Use checked arithmetic.** The compiler builtins (`__builtin_mul_overflow` and relatives), a **safe-numeric library**, or in Rust the `checked_`/`saturating_` methods rather than the bare operators — which make the overflow a value you must handle.
- **Check the operands against the limit**, as in §1 — **never the computed result**, which is the defect.
- **Prefer unsigned types for sizes and be consistent about it**, so that there is no signed-unsigned comparison in the code to get wrong in the first place — consistency here is worth more than any individual choice.
- **Enable the sanitiser.** `UndefinedBehaviorSanitizer` catches **signed overflow at runtime**, and the unsigned overflow check can be enabled separately since unsigned wrapping is well-defined and sometimes intended (Module 09 §2).
- **And treat every arithmetic operation on a value derived from input as a review trigger**, particularly a multiplication feeding an allocation. **Multiplication feeding an allocation is the single highest-yield pattern to grep for** — it is precisely the shape of the canonical defect, and the search takes minutes.

3 Why it becomes memory unsafety

AN INTEGER DEFECT IS A VULNERABILITY WHEN THE NUMBER BECOMES A SIZE

On its own, a wrapped integer is simply a wrong answer — a counter that resets, a total that is negative, a display bug. It becomes a memory safety defect when the wrong answer is used as an allocation size, a copy length, a loop bound, or an index.

So the dangerous sinks are specific and enumerable: **malloc and friends, memcpy and the string functions, array indexing, loop bounds, and pointer arithmetic**. Five sinks, which is a short enough list to audit exhaustively in a medium codebase.

Which makes this a taint analysis with a short sink list — from an input source, through arithmetic, to a size sink — and that is exactly the shape of problem a static analyser handles reasonably well (Module 07 §2), which is why integer defects are among the classes tooling genuinely finds.

And it is why integers get a module rather than a paragraph: the arithmetic is where the check is defeated, and the memory defect is merely where it surfaces — so fixing the overflow prevents the overflow, while fixing the memcpy treats the symptom and leaves the next sink exposed.

4 Type confusion

TYPE CONFUSION MEANS THE PROGRAM INTERPRETS BYTES AS A TYPE THEY ARE NOT

A downcast to the wrong class, a union read through the wrong member, a pointer cast that was never valid, or a deserialiser producing an unexpected concrete type — after which field offsets, virtual table pointers, and length fields are all read from the wrong places in memory.

Which is severe for two specific reasons. A virtual table pointer read from attacker-controlled data is control of execution, and a length field read from the wrong offset defeats every bounds check downstream of it — the checks run, and they compare against the wrong number.

And the common sources are: unchecked downcasts, tagged unions whose tag is consulted in some places and not others, deserialisation that produces whatever type the input names (Module 06), and language runtimes with dynamic typing where a JIT's type assumption can be violated.

The fix is to make the invalid interpretation unrepresentable: checked casts that fail loudly, a sum type the compiler forces you to match exhaustively on, and a parser that produces a validated domain type rather than a generic map — which is the parse-don't-validate discipline of Module 04 §4, arriving here as a type safety measure.

- Any multiplication or addition on an input-derived value that reaches an allocation — the canonical pattern from §1, and the one to search for first.
- Any comparison mixing signed and unsigned types, and any < 0 check applied to an unsigned value — the compiler will warn about both if the warnings are enabled.

- **Any assignment from a wider to a narrower integer type**, especially of something that is a length or a count — which is truncation waiting to happen.
- **Any cast between pointer types**, and **any downcast performed without a check** — both of which are greppable and both of which are usually deliberate and occasionally wrong.
- **And any union or tagged variant whose tag is read in one place and not in another**, which is the pattern behind most union-based type confusion. **All five are mechanically greppable**, and **the compiler will warn about two of them if you let it** — which is Module 12 §2's argument for turning the warnings on and treating them as errors.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Dowd, McDonald & Schuh, chapter 6	§1 and §2 exhaustively — the best treatment of C's integer behaviour in a security context. Library copy.
Wang et al. — Undefined Behavior: What Happened to My Code? (free)	§2's last point, demonstrated — real security checks deleted by real compilers, with the mechanism.
CERT C Coding Standard, the INT rules (free)	§2's practice as a rule set , with compliant and non-compliant examples for each case.
CWE-190, CWE-680, CWE-843 (free)	§1 and §4 with the shared vocabulary — use these identifiers when reporting.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write the canonical overflow defect** and confirm the small allocation with a debugger.
- 2 **Add the result-based check** and show it still passes.
- 3 **Replace it with the operand-based check** and show it rejects.
- 4 **Print the value of $-1 < 1u$** and explain it.
- 5 **Write a < 0 check on an unsigned value** and find the compiler warning.
- 6 **Write an overflow check as $a + b < a$** on signed types, enable optimisation, and see whether it survives.
- 7 **Enable UndefinedBehaviorSanitizer** on an existing project and report what it finds.
- 8 **Grep a codebase for multiplications feeding allocations** and assess each.
- 9 **Write a minimal type confusion** through a union, and observe the wrong field.

10 Replace it with a tagged sum type the compiler checks.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three integer failures and the shared consequence.
- 2 Give the canonical defect and the check that does not help.
- 3 State the rule about operands and results, and why.
- 4 What are C's promotion rules, and why is $-1 < 1u$ false?
- 5 Why might a <0 check on an unsigned value disappear?
- 6 Why is signed overflow being undefined dangerous for checks?
- 7 When does an integer defect become a memory safety defect?
- 8 Name the five dangerous sinks.
- 9 What is type confusion, and why is it severe?
- 10 Give the five review checks.